

Odabrana poglavlja iz operativnih sistema

Projektni zadatak

Pridržavati se principa objektno-orijentisanog programiranja, SOLID principa, principa pisanja čistog koda i konvencija korištenih programskih jezika. Projektni zadatak se sastoji iz dvije stavke. Prilikom predaje potrebno je predati sve *.py* fajlove, *requirements.txt* fajl u kom su navedeni nazivi i verzije biblioteka, kao i dodatne fajlove ukoliko su rađene dodatne stavke koje to zahtijevaju.

(30) Distribuirano bojenje grafa

Implementirati distribuirani algoritam za bojenje grafa koji je zasnovan na upotrebi Spark RDD API-ja. Bojenje grafa je problem u kojem se dodjeljuju boje čvorovima u grafu tako da ne postoje dva susjedna čvora koji dijele istu boju. Cilj je koristiti minimalan broj mogućih boja uz osiguranje ovog ograničenja. Dati algoritam se često koristi u problemima raspoređivanja, kao što su raspoređivanje ispita, nastave ili resursa u distribuiranim sistemima. Za rješavanje datog problema moguće je koristiti više različitih algoritama, od kojih sljedeći predstavlja dobar odnos između performansi i jednostavnosti implementacije u distribuiranom kontekstu:

1. Inicijalizovati sve čvorove kao neobojene (-1).
2. Za svaki neobojeni čvor:
 - a. prikupiti boje susjednih čvorova i
 - b. odabrati boju kandidata koju susjedi ne koriste.
3. Za čvorove sa kandidatskim bojama:
 - a. provjeriti da nema sukoba sa susjedovim bojama,
 - b. ukoliko vrijedi primijeniti boju i
 - c. ako ne vrijedi čvor ostaje neobojen.
4. Ponoviti korake 2-3 dok svi čvorovi ne budu obojeni.

(4) Implementirati klasu koja reprezentuje čvor grafa. Svaki čvor ima ID, listu susjeda i boju. Po potrebi dozvoljeno je uključiti i dodatne attribute. Implementirati funkciju za generisanje potrebnog skupa podataka. Funkcija treba da prima broj čvorova i maksimalni stepen čvora i da na osnovu toga generiše neusmjereni netežinski graf bez petlji. Implementirati Spark funkciju koja provjerava prosječan stepen čvora. Omogućiti serijalizaciju i deserijalizaciju grafa upotrebom JSON formata. Prilikom serijalizacije i deserijalizacije treba da se koriste samo osnovni atributi.

(8) Implementirati distribuirano bojenje grafa na osnovu prethodno opisanog algoritma. Prilikom izvršavanja prvo probati obojiti sa brojem boja koji je za jedan veći od maskimalnog stepena čvora. Nakon toga pokušati za manji broj boja, te pronaći minimalan potreban broj boja. Bojenje nije uspješno ukoliko se u nekom koraku ne može izabrati kandidatska boja za neki čvor (korak 2 se ne može izvršiti). U tom slučaju treba adekvatno prekinuti izvršavanje algoritma.

Napomena: Za sve operacije nad grafom potrebno je koristiti Spark RDD API! Nije dozvoljena obrada nad podacima nakon poziva `.collect()` funkcije. Sve Spark lambda funkcije treba da rade maksimalno nad jednim čvorom i njegovim susjedima (nije dozvoljeno da ulaz u lambda funkciju bude cijeli graf).

(3) Implementirati funkciju za validaciju rezultata koja se oslanja na Spark RDD API.

(2) Implementirati funkciju koja omogućava praćenje vremena izvršavanja algoritma. Izložiti CLI interfejs za pokretanje programa. Za implementaciju CLI interfejsa dozvoljeno je korištenje ugrađene `argparse` biblioteke ili bilo koje druge eksterne. Interfejs treba da omogući sljedeće funkcionalnosti:

- Specifikovanje ulazne datoteke grafa ili definisanje veličine grafa i maksimalnog stepena čvora
- Definisanje izlaznog fajla za serijalizaciju grafa
- Definisanje izlaznog fajla za rezultat bojenja

(6) Optimizovati brzinu izvršavanja algoritma. Optimizacija mora da ostvari kraće vrijeme izvršavanja u odnosu na baznu implementaciju. Prilikom optimizacije koristiti neke od sljedećih optimizacija:

- Optimizacija inicijalnog particionisanja
- Upotreba boljeg keširanja međurezultata
- Optimizacija i redukcija shuffle-inga

Obavezno je da postoji bar jedan vid redukcije shuffle-inga podataka, upotrebom redukcije `join` ili `groupByKey` operacija. Za datu modifikaciju potrebno je priložiti .pdf fajl u kom su opisane modifikacije, te priloženo teoretsko obrazloženje njihovog pozitivnog uticaja na rad algoritma. Takođe, priložiti tabelu u kojoj je upoređena brzina između 10 pokretanja algoritma na grafovima različite veličine i broja susjeda.

(7) Omogućiti inkrementalno bojenje grafa. Implementirati klasu za upravljanje promjenama koja prima promjenu u grafu i omogućava sljedeće funkcionalnosti upotrebom Spark RDD API-ja:

- detekciju obuhvaćenog podgraфа (RDD čvorova koji je obuhvaćen promjenom)
- minimizaciju broja čvorova koji trebaju ponovo biti obojeni
- ponovno bojenje redukovane liste čvorova

Promjena u grafu je definisana tipom operacije (dodavanje čvora, dodavanje veze, brisanje čvora, brisanje veze) te ID-om čvora/veze nad kojim se vrši operacija. Uporediti vrijeme izvršavanja između inkrementalnog i punog bojenja grafa nakon svake od promjena.

(20) Analiza Spotify skupa podataka

(4) Data stavka projektnog zadatka će obuhvatiti analizu Spotify skupa podataka koristeći Spark DataFrame API. Za realizaciju datog zadatka potrebno je prvo definisati tačnu šemu podataka koja koristi adekvatne tipove na osnovu opisa podataka. Validirati da se podaci adekvatno učitavaju. Za svaku učitavanu kolonu potrebno je izvršiti analizu distribucije podataka koju je zatim potrebno sačuvati u JSON formatu. Podaci i njihov opisu su dostupni na sljedećem [linku](#).

Napomena: Koristiti isključivo Spark DataFrame API za sve analize podataka!

(2) Identifikovati 10 najčešćih kolaboracija između umjetnika (dva ili više umjetnika koji nastupaju zajedno na pjesmi) i izračunajte prosječnu popularnost njihovih pesama. Za prosječnu razliku popularnosti solo pjesama manje popularnog izvođača i prosječne popularnosti kolaboracije.

(2) Pronađite "breakthrough" pjesme (popularnost >80 u albumima u prosjeku <50) i analizirajte razliku njihovih audio karakteristika (energija, plesnost, valencija) u odnosu na prosječne karakteristike u ostalim pjesmama unutar albuma.

(2) Za 5 najpopularnijih žanrova, utvrdite da li postoje optimalni rasponi tempa "sweet spot" upoređujući prosječnu popularnost pjesama unutar različitih tempa (npr. <100 BPM, 100-120 BPM, >120 BPM).

(2) Uporedite korelaciju između eksplicitnog sadržaja i popularnosti u različitim žanrovima kako biste utvrdili u kojim je žanrovima eksplicitni sadržaj pozitivan ili negativan faktor za uspjeh pjesme.

(2) Za 10 najdužih pjesama sa visokom ocjenom plesivosti (>0,8), uporedite njihov komercijalni uspjeh (popularnost) sa prosječnom popularnošću pjesama u istom žanru kako biste utvrdili da li duže trajanje utiče na uspjeh.

(2) Analizirajte da li eksplicitne i neeksplicitne pjesme pokazuju različite obrasce u valentnosti (muzička pozitivnost) i utvrdite da li je ovaj odnos dosljedan na svim razinama popularnosti (posmatrati opsege popularnosti u veličini od 0.1)

(2) Za umjetnike s najdosljednijom popularnošću (najniža standardna devijacija), analizirajte njihovu žanrovsku raznolikost kako biste utvrdili da li je dosljednost povezana sa specijalizacijom za određene žanrove.

(2) U žanrovima sa visokim akustičnim i instrumentalnim vrijednostima (>0,8), uporedite njihovu prosječnu popularnost sa vokalno teškim žanrovima kako biste utvrdili da li instrumentalni fokus utiče na komercijalni uspjeh.